FECAP – FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação

6º Semestre — Turma 6º CCOMP

SMASH

RELATÓRIO DE IMPLEMENTAÇÃO E VALIDAÇÃO DA PAREDE INSTRUMENTADA DE TREINO DE TÊNIS

Entrega 2 — Projeto Interdisciplinar: IoT e Robótica

Projeto 7 — Grupo Smash

Felipe Vallim Soares — 24026060

Guilhermy Mariano Lisboa Garcia — 23025371

Gustavo Oliveira Demetrio — 24026213

Saulo Pereira de Jesus — 24026095

João Francisco Trencher Martins

São Paulo

2026

SUMÁRIO

APRESENTAÇÃO DO PROJETO....3

1 ARQUITETURA FINAL IMPLEMENTADA....4

1.1 Blocos do sistema....5

1.2 Software de cada camada....5

1.3 Ajustes em relação à proposta da Entrega 1....7

2 CÓDIGO-FONTE ORGANIZADO E VERSIONADO....8

2.1 Versionamento....8

2.2 Como executar....9

3 INTEGRAÇÃO ENTRE SENSORES, PROCESSAMENTO E ATUADORES....10

3.1 Sensor e circuito de proteção....10

3.2 Processamento: detecção do impacto....11

3.3 Atuador: o LED da placa....11

3.4 Evidência da integração....12

4 FLUXO COMPLETO DE DADOS E AÇÕES....13

4.1 Pareamento e mensagens....13

4.2 Falhas e registro no backend....14

4.3 Demonstração com a parede real....14

4.4 Site: do treino publicado ao resultado....16

5 TESTES REALIZADOS E VALIDAÇÃO DOS REQUISITOS....17

5.1 Testes automatizados....17

5.2 Testes na bancada....18

5.3 Validação dos requisitos....19

5.4 Evidências anexas....19

6 BACKLOG FINAL ATUALIZADO....20

7 ANÁLISE CRÍTICA DAS LIMITAÇÕES E PROPOSTAS DE MELHORIA....21

CONCLUSÃO....21

APRESENTAÇÃO DO PROJETO

O treino contra o paredão é a forma mais acessível de praticar tênis, mas não gera nenhum dado. O **Smash** resolve isso com uma parede instrumentada de baixo custo. Seis alvos de MDF, em uma grade de 3 colunas por 2 linhas, têm cada um uma pastilha piezoelétrica colada atrás. Quando a bola bate em um alvo, o ESP32 identifica qual foi atingido e mede a intensidade do impacto, envia o evento ao aplicativo por WebSocket e pisca o LED da placa como retorno ao jogador. O aplicativo transforma as batidas em jogo e registra cada sessão no backend, onde ficam o histórico, as estatísticas e o ranking. Os treinos nascem no site: um treinador cria e simula, um administrador publica, o aplicativo obtém o treino pela API e o resultado de cada partida volta para o site.

Como se joga. Cada alvo tem uma de três cores. No **Color Mode**, o jogador acerta os alvos na sequência de cores pedida; no **Score Mode**, cada cor vale pontos e é preciso chegar à pontuação do nível dentro do tempo. **Infinite Color** e **Infinite Score** são as versões sem fim, que correm contra um relógio.

Este documento mostra o sistema implementado e testado com a parede real, em 03/11/2026. Cada seção corresponde a um item pedido no enunciado da Entrega 2 (Quadro 1).

Quadro 1 — Itens do enunciado e seção correspondente.

Item pedido no enunciado	Seção
Descrição da arquitetura final implementada, com os ajustes em relação à proposta	1
Código-fonte organizado e versionado	2
Evidência da integração entre sensores, processamento e atuadores	3
Demonstração do fluxo completo de dados e ações do sistema	4
Registro dos testes realizados e validação dos requisitos	5
Backlog final atualizado, com tarefas concluídas e pendentes	6
Análise crítica das limitações do sistema e propostas de melhoria	7

Vídeos do teste. Os integrantes batem nas placas e o aplicativo, no emulador Android do notebook, registra cada batida em tempo real. Os vídeos acompanham este documento na pasta de entrega e estão no Google Drive: **https://drive.google.com/drive/folders/114f-HS2OIY9HV7KM-ucOqy8qsduy_-E7?usp=sharing**

1 ARQUITETURA FINAL IMPLEMENTADA

O sistema tem cinco camadas independentes: a parede (sensores, condicionamento e ESP32), o aplicativo, o site, o backend e o banco de dados. As camadas só trocam mensagens JSON com formato definido. Por isso o aplicativo foi desenvolvido com um simulador antes de a parede existir, e depois a parede real entrou no lugar dele sem mudar nenhuma regra do jogo (Figura 1).

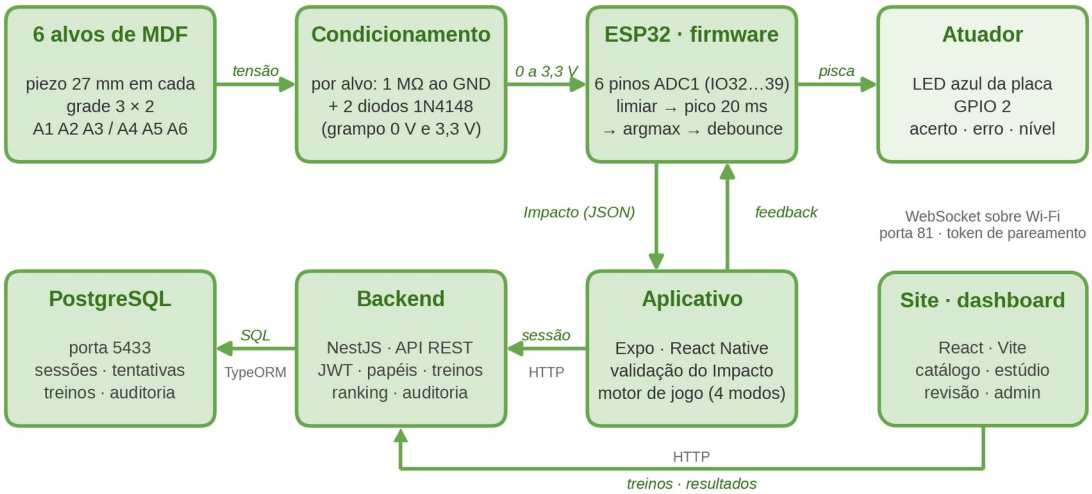


Figura 1 — Arquitetura final: hardware na linha de cima, software na de baixo.

A parede é o servidor: o ESP32 abre um servidor WebSocket e o aplicativo se conecta a ele. O aplicativo e o site são os clientes do backend, por HTTP. A parede nunca fala com o backend, então o firmware não precisa de internet, de relógio sincronizado nem de credenciais de usuário.

1.1 Blocos do sistema

Quadro 2 — Blocos da arquitetura final.

Bloco	O que foi implementado	Onde está
Sensores	6 alvos de MDF, cada um com uma pastilha piezoelétrica de 27 mm colada atrás	Placas de MDF
Condicionamento	Por alvo: resistor de 1 MΩ ao GND e dois diodos 1N4148 que limitam a tensão entre 0 V e 3,3 V	Protoboard
Processamento	ESP32 de 38 pinos; cada alvo tem o seu pino do ADC1. Firmware em C++ (Arduino)	`backend/firmware/smash_wall`
Atuador	LED azul da própria placa (GPIO 2), com um padrão de piscada para cada situação do jogo	`wall_actuator.cpp`
Comunicação	Servidor WebSocket na porta 81 do ESP32, com pareamento por token. Wi-Fi em modo SoftAP (rede própria) ou Station (rede existente)	`wall_link.cpp` `wall_protocol.cpp`
Aplicativo	Expo + React Native + TypeScript. Valida cada mensagem, conduz a sessão de jogo, devolve o retorno ao LED e joga os treinos publicados, com parada de emergência e fila offline	`frontend/src/device` `frontend/src/gameplay`
Backend e banco	NestJS + TypeORM com PostgreSQL: contas e papéis, treinos e versões, sessões, telemetria por tentativa, XP, conquistas, estatísticas, ranking e auditoria	`backend/src`
Site/dashboard	React + Vite + TypeScript: catálogo público, estúdio de treinos com simulação, revisão e publicação, histórico do jogador e administração	`frontend/web`
Implantação	Docker Compose: banco, API e site atrás do nginx	`docker-compose.yml`

1.2 Software de cada camada

Firmware. Quatro módulos (sensores, atuador, protocolo e rede), chamados a cada volta do laço principal. Nenhum deles bloqueia: não existe `delay()` no código, e toda espera compara o relógio interno (`millis()`). Assim a leitura dos sensores nunca para enquanto o Wi-Fi envia uma mensagem ou o LED pisca. Os parâmetros ficam em `config.h`: limiar por alvo, janela de pico de 20 ms, debounce de 200 ms, status a cada 2 s e limite de 256 bytes e 20 comandos por segundo.

Aplicativo. A pasta `device/` concentra tudo o que sabe que existe uma parede: o simulador, o adaptador WebSocket e o validador das mensagens. A pasta `gameplay/` tem o motor de jogo, uma máquina de estados pura (preparando → aguardando batida → resolvendo → nível concluído ou perdido, além de pausado e parede perdida), e os níveis, que são tabelas de dados. O XP é calculado pelo backend, para que nenhuma regra fique duplicada. O mesmo motor joga cada etapa de um treino publicado, conta pausas e quedas da parede, encerra a partida na parada de emergência e guarda numa fila o resultado que não chegou à API.

Site. React + Vite + TypeScript, em `frontend/web`, com a mesma conta do app e o menu conforme o papel. O visitante vê o catálogo; o treinador monta um treino como uma sequência de etapas Color ou Score, com público, objetivo, regras, riscos e dados de marketplace, e o simula no navegador com o mesmo motor de jogo do app; o administrador revisa, publica e acompanha usuários, sessões, dispositivos e a auditoria. A documentação OpenAPI da API fica em `/docs`.

Backend. NestJS em quatro camadas: domínio (regras puras, sem framework), casos de uso, infraestrutura e rotas HTTP. O banco PostgreSQL tem as tabelas `users`, `user_progressions`, `daily_streaks`, `training_sessions`, `training_session_targets` (telemetria por alvo), `training_session_attempts` (cada tentativa), `trainings`, `training_versions` e `audit_events` (auditoria), e muda só por migração. O que cada conta pode fazer depende do papel: jogador, treinador ou administrador. As rotas principais estão no Quadro 3.

Quadro 3 — Rotas principais da API.

Rota	Acesso	Função
`POST /auth/sign-up`, `/auth/sign-in`	Público	Cadastro e login; o login devolve um token JWT
`POST /me/sessions/start`	Logado	Abre a sessão: confere se o nível está liberado, marca o início e devolve um ticket
`POST /me/sessions`	Logado	Registra o resultado com o ticket e a telemetria por alvo; devolve o XP
`GET /me/statistics`, `/rankings/:categoria`	Logado / Público	Estatísticas do jogador, com o mapa da parede, e ranking
`GET /trainings`, `/trainings/:id`	Público	Catálogo e versões publicadas dos treinos
`POST /studio/trainings/:id/…` `GET /admin/…`	Treinador / admin	Simulação, revisão e publicação de treinos (só o administrador publica); painel de administração e auditoria

1.3 Ajustes em relação à proposta da Entrega 1

Quadro 4 — Mudanças entre a proposta e a implementação.

Aspecto	Entrega 1 (proposta)	Entrega 2 (implementado)	Motivo
Parede	9 alvos em matriz 3×3	6 alvos em grade 3×2	O ESP32 tem exatamente seis entradas do ADC1 que funcionam com o Wi-Fi ligado; o ADC2 lê errado com o Wi-Fi ativo.
Leitura dos sensores	Multiplexador 4051 de 8 canais e um pino direto	Um pino do ADC1 por alvo, sem multiplexador	Cada alvo funcionava sozinho; as leituras erradas só apareciam quando o multiplexador trocava de canal.
Comunicação	Bluetooth Serial no MVP, depois MQTT	WebSocket sobre Wi-Fi	Bluetooth Serial não funciona no iOS e exigiria módulo nativo. WebSocket já existe no React Native e roda no Expo Go. O MQTT é uma opção do enunciado, não uma obrigação.
Atuador	Um LED ou fita de LED em cada alvo	LED azul da placa (GPIO 2)	Atende ao requisito sem peças extras. Um buzzer e dois LEDs foram montados e retirados por escolha da equipe.
Máquina de estados	6 estados, com PARADA_SEGURA no firmware	5 estados, com uma transição de descarte	O descarte elimina batidas falsas causadas por ruído do ADC. Sem motor para parar, a parada segura ficou no aplicativo: ele trava o sistema e não envia comandos à parede.
Registro da sessão	O app enviava o resultado pronto	O servidor abre a sessão e só aceita o resultado com o ticket	Dificulta o envio de resultados inventados para o ranking.
Segurança da conexão	Não prevista	Token de pareamento, um app por vez e limite de comandos	Resultado da análise de riscos de Redes de Computadores e Cibersegurança.
Site e treinos	Não previstos	Site/dashboard com estúdio, revisão, publicação e administração; treinos pela API	Fecha o cenário do enunciado: o autor publica no site e o app executa na parede.

2 CÓDIGO-FONTE ORGANIZADO E VERSIONADO

Todo o código está em um único repositório Git: **https://github.com/2026-2-NCC6/Projeto7**. O firmware fica dentro de `backend` porque a entrega deve mostrar apenas as pastas `backend` e `frontend`; ele não entra na compilação do NestJS.

Quadro 5 — Organização do repositório.

Pasta	Conteúdo
`documentos/Entrega1`, `Entrega2`	Relatórios, slides e PDFs entregues em cada disciplina
`src/Entrega1/backend/src`	API NestJS: domínio, casos de uso, infraestrutura e apresentação
`src/Entrega1/backend/firmware`	Firmware do ESP32 e o README de montagem, calibração, protocolo e teste
`src/Entrega1/frontend/src`	Aplicativo: telas, motor de jogo (`gameplay`) e fronteira com a parede (`device`)
`src/Entrega1/frontend/web`	Site/dashboard: catálogo, estúdio, histórico e administração
`backend/test`, `frontend/**/*.test.ts`	Testes automatizados do backend e do aplicativo
`src/Entrega1/scripts`	Emulador Android, emulador da parede em rede e testes de segurança

Quadro 6 — Arquivos do firmware (`smash_wall`).

Arquivo	Responsabilidade
`smash_wall.ino`	Inicialização e laço principal, sem nenhum `delay()`
`config.h`	Pinos, limiar de cada alvo, tempos da detecção, padrões do LED e modo de rede
`wall_sensors`	Varredura dos 6 canais e máquina de estados da detecção
`wall_actuator`	Sequenciador das piscadas do LED, sem bloquear a leitura dos sensores
`wall_protocol`	Montagem do JSON do impacto e leitura dos comandos do aplicativo
`wall_link`	Wi-Fi (SoftAP ou Station), mDNS e servidor WebSocket com pareamento
`secrets_example.h`	Modelo das senhas; o `secrets.h` real fica fora do Git

2.1 Versionamento

O repositório tem mais de 100 commits, entre agosto e novembro de 2026, no padrão Conventional Commits: o prefixo diz o tipo da mudança (`feat`, `fix`, `docs`, `test`) e o escopo diz a camada (`app`, `backend`, `firmware`). As versões entregues de Cibersegurança são marcadas com tags. Senhas, token de pareamento e segredo do JWT ficam fora do Git; o repositório guarda só os modelos. O Quadro 7 mostra os marcos.

Quadro 7 — Marcos do desenvolvimento.

Data	Marco
Set. 2026	Estrutura do projeto, contas, login, os quatro modos de jogo e o simulador de parede
10/09/2026	Primeira versão do firmware, do protocolo e do adaptador WebSocket no app
14/09/2026	Telemetria por alvo no backend e tela de Estatísticas
22/10/2026	Parede 3×2 com pinos diretos, protocolo versão 2, emulador da parede e LED como atuador
24 a 31/10/2026	Pareamento por token e correções do relatório de Cibersegurança
03/11/2026	Teste com a parede real: filtro de batidas falsas e correção da reconexão do Wi-Fi
04 e 05/11/2026	Site/dashboard, treinos publicados pela API, papéis, parada de emergência, operação offline, auditoria, OpenAPI e Docker

2.2 Como executar

- **Firmware:** abrir `smash_wall.ino` no Arduino IDE (placa ESP32 Dev Module), copiar `secrets_example.h` para `secrets.h`, preencher as senhas e gravar pela USB.
- **Sistema:** em `src/Entrega1`, `npm run setup` instala as dependências e prepara o banco; `npm start` sobe o emulador Android, o backend, o aplicativo e o site, em http://localhost:5173.
- **Docker:** `docker compose --env-file .env.docker up -d --build` sobe o banco, a API e o site em http://localhost:8080.
- **Com a parede:** no `frontend/.env`, `EXPO_PUBLIC_DEVICE_SOURCE=websocket` e `EXPO_PUBLIC_DEVICE_URL` com o endereço da parede. Sem a parede, o app usa o simulador.

3 INTEGRAÇÃO ENTRE SENSORES, PROCESSAMENTO E ATUADORES

A parede usa um ESP32 DevKit de 38 pinos, 6 piezos de 27 mm, 6 resistores de 1 MΩ, 12 diodos 1N4148, 6 placas de MDF e pares trançados de cabo de rede entre os piezos e a protoboard. A Figura 2 mostra a parede montada para o teste e a protoboard.

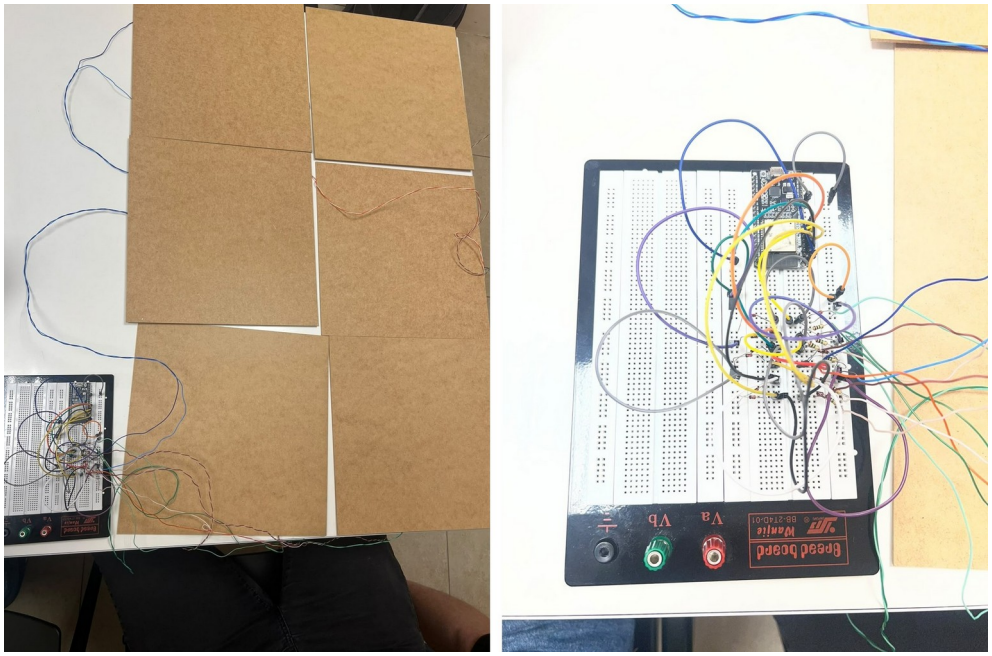


Figura 2 — Seis placas na grade 3×2 ligadas à protoboard (esquerda) e a protoboard vista de cima, com o ESP32 e os circuitos de proteção (direita).

3.1 Sensor e circuito de proteção

O piezo gera uma tensão que pode passar de dezenas de volts, e o pino do ESP32 aceita no máximo 3,3 V. Por isso cada alvo tem o circuito abaixo:

```
piezo (+) ──┬── 1 MΩ ─── GND      descarrega o piezo entre as batidas
            ├── 1N4148 ── 3,3 V   corta o que passar de 3,3 V
            ├── 1N4148 ── GND     corta o que ficar abaixo de 0 V
            └── pino do alvo (ADC1)          piezo (−) ── GND
```

Sem o resistor, o pino fica "flutuando" e lê valores aleatórios, exatamente o sintoma encontrado no alvo A1 (teste T1). O ADC tem 12 bits: converte 0 a 3,3 V em 0 a 4095, cerca de 0,8 mV por passo, e esse valor é a `intensidade` enviada ao app. Só foram usados pinos do ADC1, porque o ADC2 não funciona com o Wi-Fi ligado.

Quadro 8 — Ligação dos alvos ao ESP32.

Alvo	Posição na parede	Pino do ESP32	Cor no aplicativo
A1	Linha de cima, esquerda	IO32	Âmbar
A2	Linha de cima, centro	IO33	Azul
A3	Linha de cima, direita	IO34	Vermelho
A4	Linha de baixo, esquerda	IO36 (VP)	Vermelho
A5	Linha de baixo, centro	IO39 (VN)	Âmbar
A6	Linha de baixo, direita	IO35	Azul

3.2 Processamento: detecção do impacto

O firmware varre os seis canais sem parar. Cada batida passa pela máquina de estados do Quadro 9, implementada em `wall_sensors.cpp` com os mesmos nomes do diagrama de Teoria da Computação.

Quadro 9 — Máquina de estados da detecção.

Estado atual	Evento	Próximo estado	Ação
OCIOSO	A leitura de algum alvo cruza o limiar dele	ARMADO	Zera os picos dos seis canais
ARMADO	Imediato	MEDINDO_PICO	Começa a medir
MEDINDO_PICO	20 ms e o maior pico alcançou o limiar	EVENTO_EMITIDO	Escolhe o alvo de maior pico (argmax)
MEDINDO_PICO	20 ms e o maior pico ficou abaixo do limiar	OCIOSO	Descarta: era ruído
EVENTO_EMITIDO	Evento publicado	DEBOUNCE	Envia o impacto ao app e ao Serial, ou pisca o LED sem app
DEBOUNCE	200 ms	OCIOSO	Volta a aceitar batidas

A **janela de pico** resolve o vazamento de vibração: quando a bola bate no A5, o A4 e o A6 também vibram, mas o alvo atingido tem o maior pico. O **descarte** elimina o ruído do ADC com o Wi-Fi ligado, que abre a janela mas some antes de ela acabar; ele reduziu as batidas falsas de 11 para 0 em 20 s (T2 e T3). O **debounce** impede que a vibração que continua depois da batida conte como outra batida. Cada alvo tem o seu limiar, escolhido com o modo de calibração do firmware (cerca de 60% do pico de uma batida normal); nos testes desta entrega todos ainda usavam o valor inicial de 150.

3.3 Atuador: o LED da placa

O atuador é o LED azul do ESP32, no GPIO 2, controlado por um sequenciador separado da detecção, que não atrasa a leitura dos sensores. **Sem aplicativo conectado**, cada impacto acende o LED por 80 ms, o que mostra o

caminho sensor → processamento → atuador sem rede. **Com aplicativo conectado**, o LED só responde ao comando `feedback` do app, que conhece a regra do jogo. O mesmo retorno faz o celular vibrar.

Quadro 10 — Padrões do LED.

Situação	Quem comanda	Padrão
Impacto detectado, sem app	Firmware	1 piscada de 80 ms
Acerto	App (`acerto`)	1 piscada de 250 ms
Erro	App (`erro`)	3 piscadas rápidas de 60 ms
Nível concluído	App (`nivel_concluido`)	2 piscadas de 250 ms e 1 longa de 600 ms
Nível perdido	App (`nivel_falhou`)	4 piscadas de 80 ms

3.4 Evidência da integração

- **Sensor → processamento:** cada batida aparece no Monitor Serial como uma linha JSON com o alvo e a intensidade; em repouso, depois do filtro de descarte, nenhuma linha aparece (T3).
- **Processamento → aplicativo:** o aplicativo pareou com a parede e registrou cada batida, no alvo atingido, em tempo real (T7, T8 e os vídeos).
- **Aplicativo → atuador:** a cada jogada, o app envia o comando `feedback`, e a parede pisca o LED no padrão correspondente.

4 FLUXO COMPLETO DE DADOS E AÇÕES

O fluxo segue a sequência definida na Entrega 1: entrada → processamento → saída → registro (Quadro 11). Quando a partida é de um treino, antes da etapa 1 o app obtém pela API a versão publicada no site, e cada etapa é uma sessão aberta no servidor com o treino, a versão e o número da etapa.

Quadro 11 — Caminho de uma batida.

Etapa	Onde	O que acontece
1. Entrada	Alvo e piezo	A bola bate na placa. O piezo gera um pulso de tensão, limitado a 0–3,3 V pelos diodos.
2. Processamento	ESP32	O ADC lê o pulso. A máquina de estados escolhe o alvo de maior pico e mede a intensidade (0 a 4095).
3. Saída	ESP32 → app	O firmware envia o JSON do impacto pelo WebSocket e o imprime no Serial, que é o plano B da demonstração.
4. Validação	App (`device`)	O app confere o formato e descarta alvo inexistente, evento repetido e tempo fora de ordem.
5. Jogo	App (`gameplay`)	O motor de jogo decide se foi acerto ou erro e atualiza a pontuação e a tela.
6. Ação	App → ESP32	O app envia o comando `feedback`: o LED pisca e o celular vibra.
7. Registro	App → backend → banco	No fim da sessão, o app envia o resumo e a sequência de tentativas, com acertos, erros, tempo de reação e intensidade por alvo. O backend confere os números, calcula o XP e grava no PostgreSQL.
8. Consulta	App e site	As telas Estatísticas e Ranking do app e o histórico, a análise do treino e o painel de administração do site leem as sessões gravadas.

4.1 Pareamento e mensagens

A parede cria a rede `smash-wall` (modo SoftAP, endereço fixo `192.168.4.1`) ou entra em uma rede existente (modo Station). O aplicativo abre o WebSocket na porta 81 com o token de pareamento: com token errado, ou com outro app já pareado, a parede recusa e fecha a conexão. Aceito o token, a parede envia `hello` com a versão do protocolo e o número de alvos; se não baterem, o app mostra "Parede incompatível". Durante o jogo, a parede envia `status` a cada 2 s, e 7 s sem notícia fazem o app pausar o nível por perda de conexão. O impacto segue o contrato de dados da Entrega 1:

```
{"alvo":5,"linha":2,"coluna":2,"intensidade":1873,"t_ms":41230,"sessao":"wall-1-04821"}
```

`t_ms` é o tempo desde que a placa ligou; o app soma esse valor ao início da sessão para ter a hora real, então o firmware não precisa de relógio. A `intensidade` é relativa e só pode ser comparada entre batidas no mesmo alvo.

Quadro 12 — Demais mensagens do protocolo (versão 2).

Mensagem	Sentido	Para quê
`hello`	Parede → app	Ao conectar: versão do protocolo e número de alvos
`status`	Parede → app	A cada 2 s: sessão, clientes conectados e sinal do Wi-Fi
`erro`	Parede → app	Pedido recusado: token errado, parede ocupada ou comando inválido
`session`	App → parede	Informa o identificador da sessão de treino
`feedback`	App → parede	Aciona o LED: acerto, erro, nível concluído ou nível perdido

4.2 Falhas e registro no backend

Uma mensagem com problema nunca trava o jogo: o app a descarta e registra a falha, que aparece em Estatísticas. Quando a placa reinicia, o app recomeça a fila de eventos, porque o `t_ms` volta a zero. Não há confirmação de recebimento, de propósito: num jogo de reação, um evento atrasado é pior que um perdido.

Para um jogador logado, o backend abre a sessão antes do jogo e devolve um ticket. No fim, só aceita o resultado com esse ticket e recusa a sessão se um alvo não existir ou se repetir, se houver mais acertos que tentativas, se a intensidade sair da escala do sensor, se a soma dos alvos não bater com o total ou se a pontuação passar do máximo possível. Só então calcula o XP e atualiza nível, ofensiva diária e conquistas.

Parada segura e operação offline. O botão Parar aparece em toda partida. Tocar nele pausa o nível; escolher o motivo encerra a sessão como parada de emergência, sem XP, e trava o app até alguém confirmar Reiniciar sistema. Enquanto o app está travado, nenhum comando vai para a parede. Sem conexão com a API, o resultado fica numa fila no aparelho e é enviado quando a rede volta; sem ticket, ele entra no histórico como offline e não vale XP nem ranking. Cada um desses passos fica na auditoria.

4.3 Demonstração com a parede real

A Figura 3 mostra quadros dos vídeos do teste: a bancada completa, uma batida em uma placa e a pontuação do Infinite Score subindo no aplicativo a cada acerto.

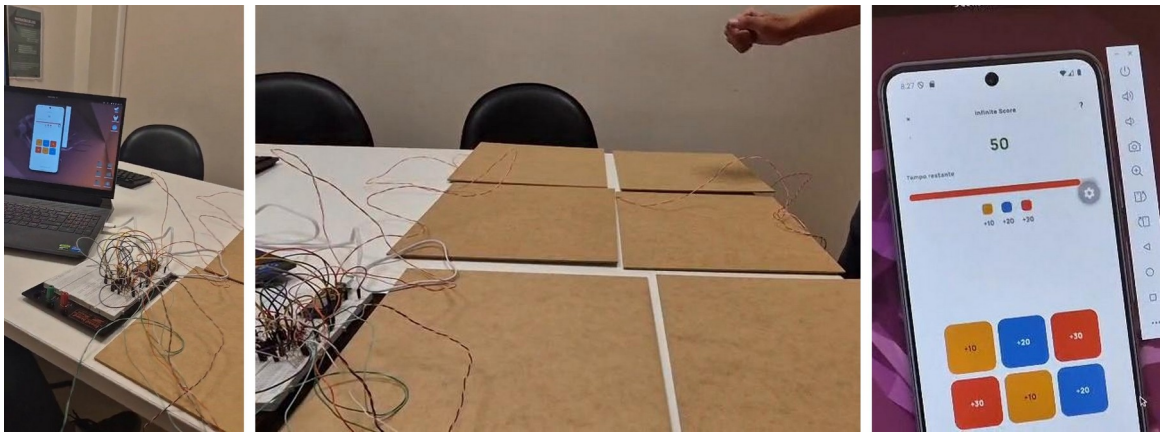


Figura 3 — Quadros dos vídeos do teste: bancada completa, batida em uma placa e pontuação no aplicativo.

Para mostrar as telas com nitidez, a Figura 4 traz prints do aplicativo no emulador Android: a tela inicial, com os quatro modos de jogo, e uma partida do Infinite Score, com a pontuação, a barra de tempo restante e os seis alvos com o valor de cada cor.

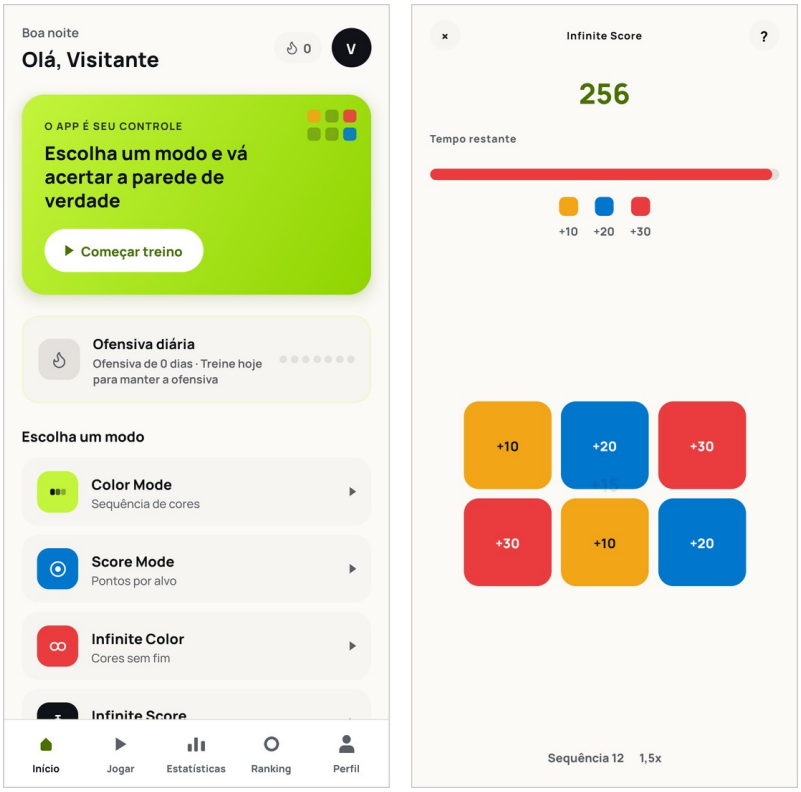


Figura 4 — Aplicativo no emulador Android: tela inicial (esquerda) e partida do Infinite Score (direita).

4.4 Site: do treino publicado ao resultado

O cenário do enunciado fecha no site. O treinador cria o treino e simula todas as etapas no navegador, com o mesmo motor de jogo do app, e envia para revisão; o servidor só aceita a revisão que foi simulada até o fim. O administrador publica, e cada publicação vira uma versão que não muda mais (Figura 5).

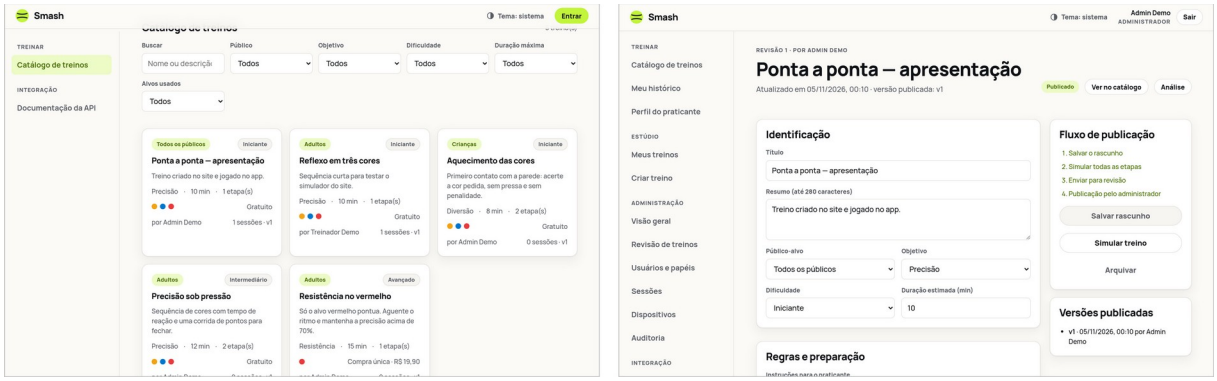


Figura 5 — Site: catálogo público de treinos (esquerda) e editor do treino com o fluxo de publicação (direita).

Depois da partida, o resultado volta para o site: o jogador vê o histórico, o treinador a análise do treino e o administrador o painel geral, com as paradas de emergência e as sessões offline (Figura 6).

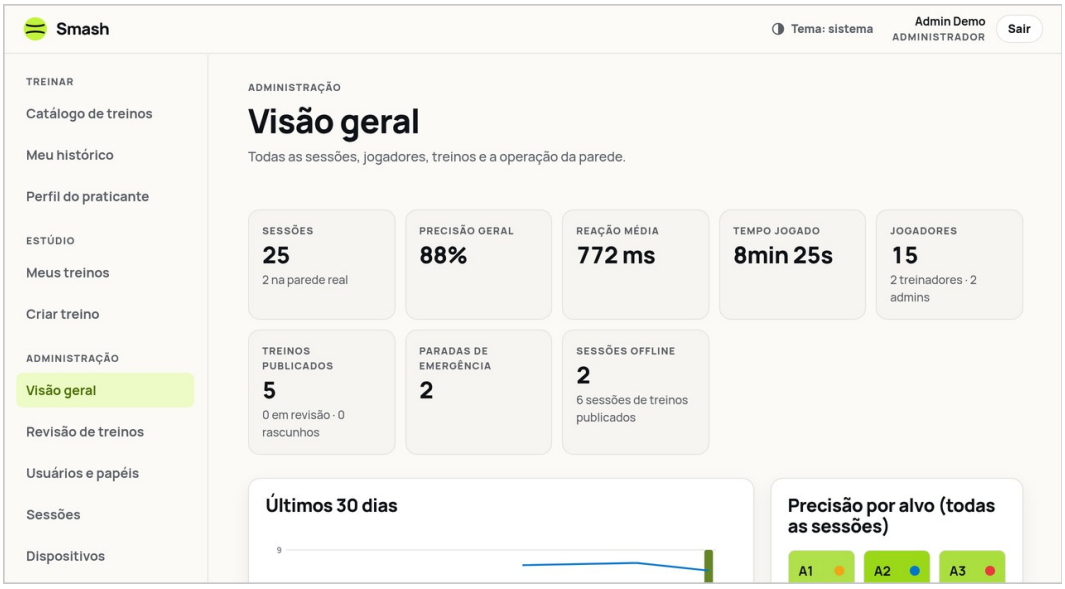


Figura 6 — Painel de administração do site com as sessões dos testes de 04 e 05/11/2026.

5 TESTES REALIZADOS E VALIDAÇÃO DOS REQUISITOS

Os testes foram feitos em três níveis. Os **automatizados** verificam as regras sem banco, rede ou hardware. O **emulador da parede** (`npm run wall`) fala o mesmo protocolo do firmware e testa o adaptador real do app pela rede. Os **testes na bancada** usam a parede real e são os únicos que testam o sensor, o circuito e o Wi-Fi de verdade.

5.1 Testes automatizados

Quadro 13 — Testes automatizados (execução de 05/11/2026).

Conjunto	Resultado	O que cobre
Backend (Jest)	18 suítes, 111 testes aprovados	Ticket de sessão, nível bloqueado, resultado aceito uma vez, telemetria que contradiz os totais, ranking, segurança do login, ciclo do treino, papéis, sessões offline e auditoria
Aplicativo (Jest)	19 suítes, 126 testes aprovados	Validação das mensagens da parede, modelo 3×2, adaptador WebSocket, queda e retorno da parede, pausa, telemetria, parada de emergência, fila offline e etapa de treino
Firmware (Arduino CLI)	Compila sem erros	Usa 73% da memória de programa
Site (TypeScript + Vite)	Compila sem erros	Tipos e versão de produção do site

5.2 Testes na bancada

Quadro 14 — Registro dos testes na bancada (03/11/2026) e de ponta a ponta (04 e 05/11/2026).

#	Teste	Resultado	Ação
T1	Leitura em repouso, sem ninguém batendo	A1 chegou a 176, A6 a 128 e A5 a 71; o limiar é 150	Revisar a ligação do A1 (pendente)
T2	Firmware do projeto em repouso por 20 s	11 batidas falsas, 9 delas no A1	Descarte da janela cujo pico fica abaixo do limiar
T3	Repetição do T2 após a correção	0 batidas falsas em 20 s	Aprovado
T4	Rede própria da placa (SoftAP)	Rede WPA2 criada pela placa, parede em `ws://192.168.4.1:81`	Aprovado; é o modo da apresentação
T5	Placa entrando em uma rede existente (Station)	Troca de chaves recusada; o firmware reiniciava a tentativa a cada 3 s	Senha corrigida e reconexão entregue ao driver do Wi-Fi. Aprovado
T6	Pareamento pelo notebook, com o token	`hello` com protocolo 2 e 6 alvos, `status` a cada 2 s, sinal de -56 dBm	Aprovado
T7	Pareamento do aplicativo no emulador	Serial da placa: "cliente 0 pareado"	Aprovado
T8	Partida com batidas reais no Infinite Score	Cada batida apareceu no aplicativo em tempo real, no alvo atingido	Aprovado (vídeos 1 e 2)
T9	Treino criado, simulado e publicado no site	3 treinos simulados até o fim e publicados, com auditoria	Aprovado
T10	Treino publicado jogado no app	6 sessões com treino, versão e etapa	Aprovado
T11	Parada de emergência na partida	2 paradas, sem XP e com o motivo registrado	Aprovado
T12	Resultado sem conexão com a API	2 sessões enviadas depois, marcadas como offline	Aprovado

Nas batidas falsas (T1 a T3), uma leitura isolada de ruído bastava para abrir a janela, e o firmware enviava o evento mesmo com o maior pico em 44. O descarte corrige o efeito; a causa física no A1, provavelmente o resistor de 1 MΩ mal encaixado, continua no backlog.

5.3 Validação dos requisitos

Quadro 15 — Situação final dos requisitos da Entrega 1.

Requisito	Situação	Evidência
RF01 — Aquisição dos sensores e envio do impacto	Atendido com ajuste	Lê 6 sensores, um pino do ADC1 por alvo, sem multiplexador. Testes T2, T3 e T8.
RF02 — Comunicação sem fio	Atendido	WebSocket sobre Wi-Fi, nos modos SoftAP e Station. Testes T4 a T7.
RF03 — Gamificação	Atendido	Quatro modos, 20 níveis no Color e no Score, pontuação em tempo real e mapa de acertos da parede. Teste T8.
RF04 — Parada segura	Atendido no app	Botão Parar: encerra sem XP, registra o motivo e trava o app até Reiniciar sistema (T11). PARADA_SEGURA não foi feita no firmware.
Atuador (RF08 do enunciado)	Atendido	LED da placa com cinco padrões (Quadro 10), comandado pelo firmware e pelo app.
Site e treinos (RF14 a RF18 do enunciado)	Atendido	Estúdio, simulação, revisão, versões, catálogo pela API e administração (T9 e T10).
Offline e auditoria (RF21 e RF23 do enunciado)	Atendido	Fila offline sem XP nem ranking (T12); ações críticas em audit_events.
RNF01 — Varredura em menos de 1 ms	Atendido no projeto, sem medição	Laço sem bloqueio e varredura O(6). Não medimos com instrumento; as batidas apareceram sem atraso perceptível.
RNF02 — Tolerância ao vazamento de vibração	Atendido	Maior pico na janela de 20 ms e descarte do ruído (T3 e T8). O vazamento não foi medido em números.
RNF03 — Padronização e tipagem	Atendido	Contrato `Impacto` tipado em TypeScript e validado no app; mensagem inválida vira falha registrada.

5.4 Evidências anexas

Os arquivos do Quadro 16 acompanham este documento na pasta de entrega. Os vídeos também estão no Google Drive: **https://drive.google.com/drive/folders/114f-HS2OIY9HV7KM-ucOqy8qsduy_-E7?usp=sharing**

Quadro 16 — Arquivos de evidência.

Arquivo	Conteúdo
`Smash-Embarcados-Entrega2-Video1.mp4`	24 s. Batidas nas placas e a pontuação subindo no Infinite Score.
`Smash-Embarcados-Entrega2-Video2.mp4`	23 s. Login, escolha do Infinite Score e batidas com a tela respondendo.
`Foto-1-Protoboard-ESP32.jpg`, `Foto-5-Protoboard-Vista-Superior.jpg`	ESP32 e os circuitos de proteção na protoboard.
`Foto-6-Parede-3x2-Vista-Geral.jpg`	As seis placas na grade 3×2, ligadas à protoboard.
`Foto-2-Placas-MDF.jpg`, `Foto-3-Teste-Bancada.jpg`	Integrantes batendo nas placas durante o teste.
`Foto-4-Emulador-Notebook.jpg`, `Print-1-Tela-Inicio.png`, `Print-2-Score-Mode.png`, `Print-3-Infinite-Score.png`	Aplicativo no emulador Android do notebook.
`Print-4` a `Print-6-Site-*.png`	Site: catálogo, editor e painel do administrador.

6 BACKLOG FINAL ATUALIZADO

Quadro 17 — Backlog inicial (Entrega 1).

ID	História	Situação
US01	Ligar um piezo na protoboard com o circuito de proteção	Concluída
US02	Colar o sensor no MDF e ler o sinal no ESP32	Concluída
US03	Montar dois alvos lado a lado e isolar o vazamento de vibração	Concluída com as seis placas; vazamento tratado pelo maior pico, sem medição numérica
US04	Integrar o multiplexador ao ESP32 para ler 9 alvos	Substituída: um pino do ADC1 por alvo, com 6 alvos
US05	Programar a máquina de estados e imprimir o JSON no Serial	Concluída, com a transição de descarte
US06	Fazer o app ouvir a parede e mostrar os eventos	Concluída com WebSocket no lugar de Bluetooth/MQTT
US07	Mostrar na tela o alvo que o jogador acabou de acertar	Concluída
US08	Criar o banco PostgreSQL e as rotas do NestJS para salvar o histórico	Concluída

Quadro 18 — Itens que entraram durante o desenvolvimento (concluídos).

Item	Camada
Cadastro, login, perfil, tema claro/escuro, XP, ofensiva diária e conquistas	App e backend
Quatro modos de jogo e 40 níveis configuráveis	App
Simulador de parede no app e emulador de parede em rede	App e scripts
Pareamento por token, Wi-Fi SoftAP e Station, limite de comandos	Firmware
Retorno pelo LED da placa e vibração do celular	Firmware e app
Sessão aberta pelo servidor (ticket) e validação dos resultados	Backend
Estatísticas com telemetria por alvo e ranking	Backend e app
Mitigações do relatório de Cibersegurança	Todas
Filtro de batidas falsas e reconexão do Wi-Fi	Firmware
Site/dashboard, treinos no app, parada de emergência, operação offline, auditoria, OpenAPI e Docker	Site, app e backend

Quadro 19 — Itens pendentes.

Item	Prioridade	Observação
Calibrar o limiar de cada alvo	Alta	Todos usam 150; uma batida real chegou a 159, então batidas fracas podem escapar.
Revisar a ligação do A1	Alta	Ruído em repouso acima do limiar (T1).
Fixar as placas na parede, com isolamento	Alta	Os testes foram feitos com as placas sobre a mesa.
Contar 20 batidas por alvo na bancada	Alta	Dá a taxa de detecção para o relatório final.
Testar em um celular físico	Média	O teste foi feito no emulador Android.
API e site publicados com HTTPS	Média	O Docker já sobe o banco, a API e o site, mas tudo ainda roda na máquina da equipe.
Integração com máquina lançadora ou simulador	Baixa	Fora do escopo do protótipo.

7 ANÁLISE CRÍTICA DAS LIMITAÇÕES E PROPOSTAS DE MELHORIA

O protótipo transforma uma batida em um evento confiável, usado em jogo e registrado no banco. Acertaram as decisões de separar a parede das regras de jogo, escolher o alvo pelo maior pico, usar um pino por alvo e dar à placa uma rede própria. O Quadro 20 lista o que limita o uso fora da bancada.

Quadro 20 — Limitações e propostas de melhoria.

Camada	Limitação	Efeito	Proposta
Hardware	Placas testadas sobre a mesa	Vibração e rebote mudam na parede; o isolamento não foi validado	Fixar com espaçadores de espuma e repetir T2, T3 e T8 na parede
Hardware	Protoboard com fios soltos	Mau contato gera ruído (caso do A1) e a montagem é frágil	Placa perfurada ou PCB, com um conector por alvo
Hardware	Um único atuador, pequeno	De longe, o jogador quase não vê o LED da placa	LED ou fita em cada alvo, ou buzzer, com o mesmo comando `feedback`
Firmware	Limiar igual e intensidade relativa	Batidas fracas podem escapar; a força não é comparável entre alvos	Calibrar cada alvo e normalizar a intensidade por alvo
Comunicação	Dependência da rede local	Em SoftAP, só a rede da placa alcança o backend; em Station, a rede precisa ser 2,4 GHz	Backend publicado com HTTPS e sessão guardada no app para envio posterior
Comunicação	Token sem criptografia (`ws://`)	Quem está na mesma rede vê o token	Protegido pela senha WPA2 da rede; `wss://` pesa demais no ESP32
Software	Acertos informados pelo app	O backend confere a consistência, mas confia nos acertos	A parede assinar os impactos para o backend conferir
Escopo	Sem máquina lançadora	O treino vai do site à parede, mas nenhum comando de máquina é executado	Simulador da máquina ou interface autorizada pelo fabricante

Quadro 21 — Próximos passos.

Prazo	O que fazer
Antes da apresentação	Revisar o A1, calibrar os limiares, voltar o firmware para SoftAP, fixar as placas e repetir T2, T3 e T8. Manter o Monitor Serial pronto como plano B.
Depois	Medir a taxa de detecção por alvo, testar em celular físico, publicar a API e o site com HTTPS, placa definitiva e LED por alvo.

CONCLUSÃO

O Smash funciona de ponta a ponta na bancada: o treino publicado no site chega ao aplicativo, a batida vira um evento no ESP32, o app conduz o jogo, aciona o LED e registra a sessão, e o resultado volta para o site. Falta calibrar e fixar a parede e publicar o sistema com HTTPS.